

A W3C PROV–Aligned Metamodel for Tracing End-to-End Provenance in ML Pipelines^{*}

Ahmad Qadeib Alban¹[0000–0002–4649–5219], Khalid Belhajjame¹[0000–0001–6938–0820], and Daniela Grigori¹[0000–0003–1741–8676]

PSL - Paris-Dauphine University
{ahmad.qadeib-alban,kbelhajj,daniela.grigori}@dauphine.psl.eu

Abstract. Once specified and enacted, end-to-end machine learning (ML) pipelines, together with contextual information about how artifacts are consumed and produced during execution, constitute valuable assets that can be shared or published for reuse. Indeed, ML pipelines involve multiple repetitive data transformation steps before, during, and after model training, and evaluating alternative configurations at each stage requires continuous monitoring of the workflow, particularly to support informed model selection for deployment. Although existing monitoring and experiment-management solutions record metrics and configurations, these logs are typically expressed using ad hoc data models that capture only limited relationships between artifacts and pipeline steps, and lack support for tracing complete derivation paths from training data to deployed models. To overcome such limitation, we propose, in this paper, a unified, W3C PROV-compatible metamodel for capturing end-to-end provenance of ML pipelines. We validate the proposed metamodel through a pipeline for detecting challenging behaviours in children with autism spectrum disorder from wearable physiological signals, and we illustrate its usefulness through decision-support queries that relate model behaviour and evaluation metrics to upstream data transformations, configurations, and execution environments, enabling systematic model selection, debugging, and governance across the ML lifecycle.

Keywords: Provenance · Machine Learning Pipelines · Data Learning Preparations · W3C PROV · MLflow

1 Introduction

The development of machine-learning models is inherently exploratory and iterative, spanning data ingestion and preprocessing, feature engineering, model design, training, and optimisation. Because model performance depends fundamentally on the input and training configurations, understanding a pipeline requires reasoning over the end-to-end flow rather than isolated steps. The resulting large

^{*} The source code, experimental artifacts and datasets supporting this work are publicly available at https://github.com/aq1992dauphine/End_To_End_Provenance_Meta_Model

volume and heterogeneity of artifacts (datasets, code, configurations, metrics) pose a significant challenge for researchers and data analysts, both in archiving results and metadata locally, and in sharing information among collaborators or more broadly with the community as a whole when a study is published [11]. These demands motivate principled, lifecycle-wide provenance that connects what was intended to what actually ran, and that supports representative queries over data transformations, run configurations, and outcomes.

Provenance, in this context, refers to the documentation of how data, models, and intermediate artifacts are produced, transformed, and used throughout the ML lifecycle. Within a precise and efficient provenance framework, a clear distinction between prospective lineage and retrospective lineage is highly required. Prospective lineage describes the intended structure of the pipeline, the workflow blueprint defined at design time, which specifies which steps should occur, in what order, and with what expected inputs and outputs [3]. By contrast, retrospective lineage captures the concrete execution history: the actual steps that were run, the datasets they consumed and produced, the parameters and environments used, and the artifacts generated [9].

Nevertheless, to the best of our knowledge, existing provenance approaches frequently fail to provide a comprehensive, end-to-end view of data lineage across the ML lifecycle. This is widely attributed to the fact of granularity mismatches. Some solutions adopt coarse-grained metadata logging, capturing only high-level experiment or run context, whereas others record data derivations at attribute level, which is required to explain specific value changes. However, these fine-grained tools typically focus on the pre-processing stages. Taken together, these limitations motivate the need for a unified data model capable of integrating provenance information across all stages of a heterogeneous ML lifestyle.

Building upon and going beyond these efforts and limitations, we present a modular provenance meta-model that supports the full lifecycle of ML experiments by enabling comprehensive, end-to-end data analysis. Our model organises the ML lifecycle into three phases (detailed later in the paper and depicted in Fig.1): data curation, which curates raw data; learning data preparation, which prepares curated data for learning; and learning, which trains and evaluates models [12].

As a running example, we consider an ML pipeline designed for detecting challenging behaviours in children with autism spectrum disorder (ASD) using physiological signals (heart rate, EDA, etc.) captured by wearable sensors. Processing such data imposes complex chained transformations, including resampling heterogeneous sensors data collected at different rates, outlier removal, balancing classes, and expert annotation, making automation highly desirable for early recognition of severe behaviours and prevention of self-harm. Yet, the manifestation of these behaviours varies markedly across children, raising substantial challenges on ML algorithms and demanding specialized expertise to prepare, clean, and interpret the data [1]. In practise, interdisciplinary teams of clinicians,

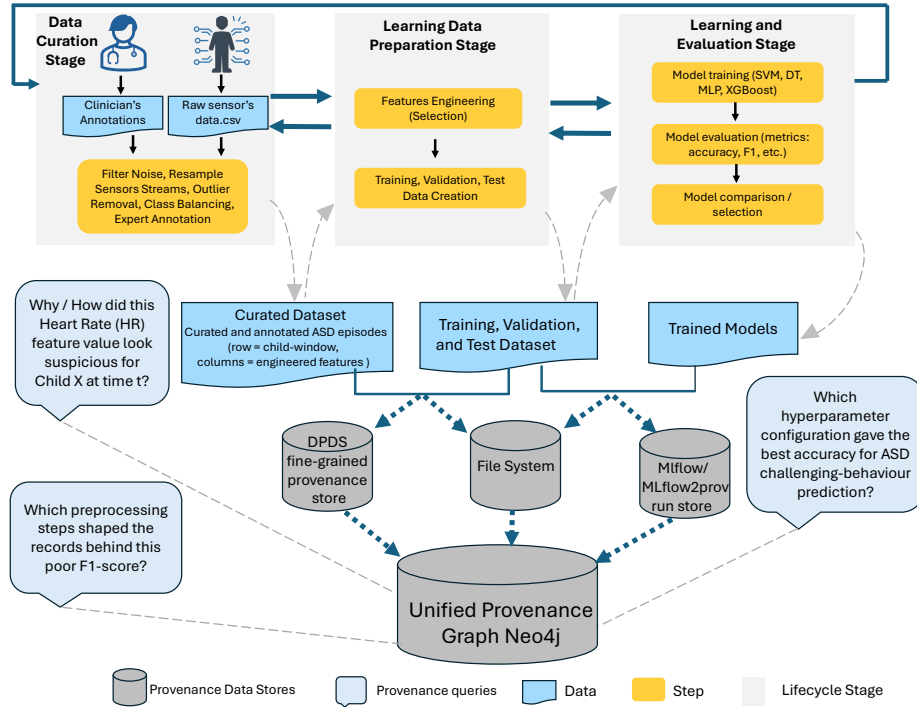


Fig. 1: Summarised example of provenance tracking for the ASD challenging-behaviour detection pipeline, highlighting its main steps and stages, from data curation through learning data preparation to model training and evaluation, together with provenance storage layers, and unified dataflow used to answer representative queries (Tables 1-3).

behavioural therapists, data scientists, and ML engineers often decompose the problem so that each group addresses a different facet, with one team consuming data generated by another. While this decomposition makes the problem tractable, it creates a new challenge: analysing the data and its transformations in an integrated way.

A unified view of the entire lifecycle, spanning multiple steps, stages, and personas, is therefore required. Using the ASD pipeline in Fig. 1 as a motivating example, we later show how fine-grained curation provenance and run-level learning provenance can be integrated into a unified provenance store to support end-to-end analysis and decision-support queries. Although our motivating example is drawn from healthcare, similar requirements arise in other domains that rely on complex computation experiments executed on heterogeneous high-performance computing (HPC) infrastructure and involving collaboration among diverse experts.

Accordingly, the contributions of this paper are:

1. **A unified, W3C PROV-compatible provenance meta-model** that supports the full lifecycle of ML experiments. The meta-model provides a generic data flow representation that explicitly separates the intended design (Prospective Lineage, e.g., Pipeline, Stage, Step) from the recorded execution history (Retrospective Lineage, e.g., Pipeline Run, StepRun), enabling the linkage between "what was planned" and "what actually happened".
2. **A suite of competency provenance queries** that traverse the entire ML pipeline, associating raw input data to final model outcomes.
3. **Empirical evaluation** that showcase the utility of the meta-model using a real-world application scenario involving detecting challenging behaviors in children with Autism Spectrum Disorder (ASD) from wearable physiological signals.

This paper is structured as follows. Section 2 reviews related work on provenance in ML pipelines. Section 3 introduces the proposed unified meta-model. Section 4 maps the meta-model to two prominent provenance-capture solutions that cover complementary parts of the ML lifecycle: DPDS for data preprocessing provenance [4]) and MLflow2PROV for model lifecycle and experiment provenance [11]). Section 5 presents the competency provenance queries supported by the meta-model. Subsequently, we describe the workflow, design, and the implementation of the proposed framework 6. Section 7 reports the experimental evaluation, with a particular focus on the expressiveness and utility of the unified provenance model, as well as the runtime performance of the implementation. Finally, Section 8 concludes the paper and outlines directions for future work.

2 Related Work

Provenance management in machine learning (ML) pipelines is an increasingly studied area, focusing on ensuring reproducibility, enhancing explainability, and facilitating debugging [11,8,5]. In our analysis of the literature, we find that related efforts are structured around database provenance, dedicated ML metadata management, granular data transformation tracking, and end-to-end lifecycle systems.

Focusing on relational databases, we observe that most proposals use query instrumentation or rewriting to propagate provenance attributes to the query output. This approach demonstrates how output data is derived from input data through relational operations. Systems such as Perm [7], GProm [2], and older tools like Smoke [10] exemplify such proposals, being optimized for capturing lineage of relational operators (e.g., ASPJ queries and set operations). Nonetheless, these systems are unsuitable for many modern data science pipelines because they assume queries are dedicated to relational operators and are tightly coupled to SQL. In contrast, by abstracting transformations within our meta-model to core operators and capturing provenance via observed data changes, our approach covers an extensible set of data preparation operators, many of which extend

beyond relational algebra. This makes it more flexible for the "open world" of data pre-processing [4].

A fundamental shift occurred in provenance solutions with the introduction of ML Experiment Management Systems (EMS) such as MLflow [5], ModelDB [13], ModelKB [6], and Ease.ML [8]. In particular, this shift moved the focus from the pure data derivation problem, addressed by aforementioned database provenance systems, which are limited to relational operations, toward the unique challenges inherent to the exploratory and iterative lifecycle of machine learning (ML) pipelines. This evolution was necessary because of the optimization nature of ML development, which revolves around constant experimentation to optimize metrics like accuracy, requiring systematic tracking of numerous configurable parameters, code versions, and training processes for reproducibility, often through integrations with specific frameworks such as scikit-learn or SparkML [5].

However, EMS solutions mainly record metadata at a coarse level. In contrast, fine-grained approaches, like that proposed by Chapman et al. (DPDS), track data derivations at the attribute level. This level of detail is necessary for precisely inspecting data transformations, but these methods often stop before model selection. We argue that robust end-to-end data analysis in ML pipelines requires both coarse and fine-grained provenance, motivating their integration into a unified model.

Fine-grained lineage is especially important because data transformations can unintentionally introduce bias or affect fairness, such as by altering feature distributions. Assessing these causal effects requires tracing changes at the element level, something that coarse-grained frameworks cannot achieve.

Conversely, coarse-grained provenance (model and evaluation metrics tracking) is sought for assessing which preparation steps and configurations produce the desired predictive performance. In summary, while existing approaches address portions of the problem, none provide a unified and comprehensive solution. Our work fills this gap by introducing a framework that integrates these capabilities end-to-end.

3 A MODULAR META-MODEL FOR ML PIPELINE

In this section, we introduce the architecture of our meta-model, which provides a generic data-flow representation for the ML lifecycle. This framework is structured around three phases. (I) Data curation, where the gap between raw data and data that is suitable for consumption (e.g., as input to train ML models) is addressed. This phase spans handling the heterogeneous nature of the data (for example, in the ASD use case, wearable-sensor streams may be stored in different file systems or DBMSs), developing a data-intensive script to clean, filter, and validate the data, and annotating it with domain-specific knowledge (e.g., challenging vs. non-challenging behavior labels). Without these processes, it would be nearly impossible to make the data useful for ML algorithms. (II) Learning data preparation phase, which consists of selecting and shaping relevant subsets of the curated data to be used for learning. For instance, when the ML

task aims to classify physiological features into challenging and non-challenging behaviours, feature selection/engineering is required to retain those attributes that have the greatest impact on model performance. In the same vein, model performance is strongly influenced by how data instances are split into training, validation, and test subsets. Such splitting operations are typically represented as a data-reduction operations that filter rows (and, in some cases, columns) without altering the schema of the curated dataset. (III) Learning, which encompasses training, validation, and evaluation processes, including optimizing learning parameters (e.g., number of epochs, learning rates) and monitoring the evolution of evaluation metrics.

The model architecture, conveyed as a unified class diagram in Fig. 2, can be more easily understood when instantiated with the concrete ASD use case depicted in Fig. 1. From an operational standpoint, a classification pipeline may contain several instances of each stage, where each stage consists of a series of chained data transformations that consume and produce data (the Step class). A Step is specified by a set of input and output parameters (InputArtifact/OutputArtifact), associated with the abstract ParameterArtifact class, which characterizes the input and output specifications of the step operations.

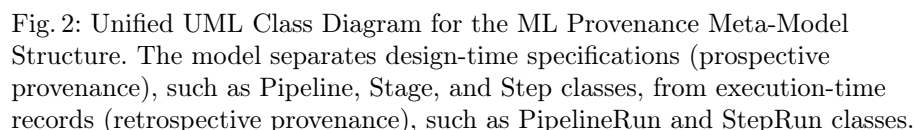
3.1 Core Concepts of the Meta-Model

For clarity, we present the main classes constituting the schema of our model from the perspective of prospective and retrospective aspects. Crucially, the architectural foundation of the proposed meta-model clearly distinguishes between the design blueprints (prospective) and the executed history (retrospective), a standard convention in robust data lineage systems.

- Prospective Lineage: defines the intended workflow structure or blueprint ("what was planned"), typically specified at design time. This workflow structure is encoded by Pipeline, Stage, and Step classes, where Pipeline provides the container boundary across different phases (i.e, stages). Data flow is declared via Datalink, which connects step-level input/output specifications (including expected schema and role), while static configuration is captured as ParameterArtifact, and declared I/O is represented by InputArtifact and OutputArtifact classes.

In the ASD challenging-behaviour detection pipeline (Section 1; Fig. 1), the prospective lineage specifies the intended curation and preparation steps and the experimental plan (candidate learners and their hyperparameters grids) prior to execution.

- Retrospective Lineage: captures the concrete execution history, recorded as PipelineRun and StepRun, together with detailed run-time logs about the computational tasks and the environment used. Each StepRun links back to its design-time Step and to the enclosing PipelineRun, and binds actual inputs and outputs as instances of Artifact and its specialisations. More precisely, the abstract Artifact class unifies all data objects consumed and produced by



In the ASD pipeline, retrospective lineage records the concrete preprocessing choices and learning configurations actually executed (e.g., the selected normalization/outlier strategy, the chosen hyperparameter setting), together with the resulting trained model artifacts and evaluation metrics

As discussed in Section 1, achieving end-to-end traceability in ML pipelines requires bridging the granularity gap between fine-grained data-preparation

provenance and coarse-grained experiment/run provenance. This section operationalizes that integration within our meta-model by aligning two leading provenance-capture solutions.

It is worth noting that the rationale for choosing these two solutions, is that they represent complementary granularities required to cover the entire end-to-end ML lifecycle. This selection directly addresses the limitation that existing provenance systems, when used independently, fail to provide a comprehensive, unified view across all stages of the pipeline.

- Chapman *et al.* [4], refereed as DPDS, who provide fine-grained, attribute-level provenance for granular data preparation and transformation operations.
- Schlegel *et al.* [11], whose MLflow2PROV tool builds upon MLflow [5] to capture row-level/run-level provenance traces of ML experiments, their artifacts, and relationships.

4.1 Granularity-Specific Mapping Across Stages

In view of the limitations mentioned earlier, it is compulsory to make sure that the design of our meta model accommodates the distinct focus and granularity of the integrated solutions (DPDS and MLflow/MLflow2PROV) across the sequential pipeline stages. To this end, we show how the design of our meta-model accommodates the semantics of these solutions:

First, the Data Curation stage is primarily supported by Chapman *et al.* (DPDS’s fine-grained data preparation). In this phase, meta-model classes such as DatasetArtifact, DataRecordArtifact, and DataCellArtifact are mapped directly to DPDS’s attribute-level semantics and formalized categories of operators (i.e., Data Reduction, Augmentation, Transformation, and Fusion), detailing derivations down to individual data cells, for which MLflow/MLflow2PROV has no native Coverage.

Next, since the Learning Data Preparation stage acts as a bridge, the classes Step and StepRun classes capture operations like splitting and feature selection, utilizing DPDS’s precise reduction/selection semantics, while MLflow/MLflow2PROV encodes the resulting splits (e.g., trainDS, validDS, testDS) as dataset-level artifacts, thereby producing row-level lineage for the partitioned data artifacts. Finally, since the Learning stage focuses on execution history and coarse-grained metadata, the classes PipelineRun, StepRun, ModelArtifact, and Evaluation-Metric correspond directly to the run/parameter/metric/model artifact tracked by MLflow2PROV, as the DPDS framework concentrates mainly on pre-processing and has counterparts for these execution-level learning components.

4.2 Integration Methodology: Bridging the Granularity Gap

We now turn from stage-wise alignment to its execution, in that we introduce our integration methodology, which binds fine-grained curation provenance with run-level learning lineage.

- The linking Mechanism: A critical statement is the `record_id` attribute, which is added to the `DatasetArtifact` class. This ID serves as the linking mechanism, functioning as the primary key in the fine-grained provenance derived from DPDS (Data curation) and as a foreign key linking to the datasets consumed by the downstream learning stages. Concretely, during the Learning Data Preparation (e.g., when splitting into train/validation/test), an association function queries the curation-provenance store to retrieve, for each output row, the `record_id` shared by its attribute-level provlets; the identifier is subsequently attached to the corresponding rows into the split datasets. Because curation provenance is captured at attribute level, the lookup aggregates across a record’s attributes and selects the common `record_id`. Maintaining this identifier alongside the split datasets yields a deterministic, auditable join key that links subsequent `stepRuns`, `ModelArtifacts`, and `EvaluationMetrics` back to the exact records, and by composition, to the contribution cells, in the curation lineage.
- Mapping the `record_id` across stages ensures that the `wasDerivedFrom` relations created during preprocessing remain traceable through the `DatasetArtifact` objects produced in both the Learning Data Preparation stage and the Learning stage.
- In parallel, the `DataLink` class reinforces this mechanism by explicitly representing the intended data-flow channels between step specification and their corresponding executions. Once these prospective links are specified and aligned with the retrospective execution traces, a coherent end-to-end view of lineage becomes apparent, from the raw input data, through intermediate datasets and transformations, to the final `ModelArtifact` and its associated `EvaluationMetrics`.

5 Querying Provenance

Once the meta-model is instantiated and populated across the Data Curation, Learning Data Preparation, and Learning stages, provenance questions reduce to graph traversals over standard PROV relations such as `used`, `wasGeneratedBy`, and `wasDerivedFrom`. By applying this representation to our use case of detecting challenging behaviours in children with autism (ASD) described in section 1, we can show how different classes of queries recover fine-grained derivations, cross-stage paths, and configuration-level explanations, yielding comprehensive traceability from raw sensors data to model metrics. From an operational standpoint, our prototype realizes these traversals as Cypher queries over a Neo4j graph store via q query generator. Tables 1, 2, and 3 catalogue representative query patterns for, respectively, data curation, integrated data-to-model reasoning, and learning/configuration analysis; in the remainder of this section we discuss representative instances from each group in the context of the ASD pipeline.

Table 1: Provenance queries for fine-grained data transformations in the Data Curation stage

Provenance Query	Input / Focus	Alignment with End-to-End Pipeline
Why-provenance	Individual item $(d_{i,a})$	data Traces the raw input elements that directly influenced this value.
How-provenance	Individual item $(d_{i,a})$	data Identifies the input data and sequence of operations that produced the value.
Item-level feature operation	Individual item $(d_{i,a})$	data Lists the specific transformations applied to this value during the curation stage.
All transformations	Dataset D	Returns all operations applied to the dataset and the features they affected.
Feature transformations over D	Dataset D	Summarises the feature-transformation steps, including those implemented as serialized preprocessing objects.

5.1 Queries Focused on Data Transformation and Origin

A representative example from Table 1 is the why-provenance query for an individual data item. Within the ASD use case (Sect. 1), a clinician might observe an abnormally high value for a certain physiological signal (i.e, electrodermal activity (EDA)) for a given child and wish to determine whether this spike is natural or the result of a preprocessing operation. The corresponding query starts from the DataCellArtifact representing the EDA value $(d_{i,a})$ in the curated dataset and traverses backwards over `wasDerivedFrom` and `wasGeneratedBy` relationships to the DataRecordArtifact and the sequence of StepRuns, producing it. The StepRun might be resampling sensor readings, substituting missing values, or outlier removal operation. Thus, each StepRun would be typed by the underlying operator templates $(\tau, \sigma/\pi, \alpha, join, append)$. This way, the data scientist can reconstruct exactly which transformations affected that value, with links to the original DatasetArtifact of raw sensor readings.

5.2 Integrated Queries Linking Data to Model Performance

This type of query requires integrating fine-grained curation provenance with run-level learning lineage to answer questions such as the "Tracing Records to Metrics" query, which illustrates how integrated provenance links model performance back to specific episodes in the autistic dataset. This query will be useful in a case where the team trains a classifier for the recognition of challenging behaviours and observes that the F1-score for a particular child is significantly poor. To figure out the reason behind this weak performance, the corresponding EvaluationMetric node for that run would be the start point for the query that follows `wasGeneratedBy` to the evaluation StepRun, then via the `used` relation to the DatasetArtifact and DataCellArtifacts corresponding to that child's episodes. From there, the traversal continues upstream into the corresponding curating graph, revealing which filtering, resampling, class-balancing, and annotation StepRuns shaped those records. This path allows the analysts to determine, for example, whether misclassifications are driven by noisy Heart-Rate (HR)

Table 2: Integrated provenance queries linking fine-grained data transformations to model performance.

Provenance Query	Input / Focus	Alignment with End-to-End Pipeline
Which dataset trained a given model, and what is its origin?	Trained model	Traces the model back to its training dataset and further to the curation steps and raw data from which it was derived.
How was the training data of a specific model processed?	Trained model	Follows links from the model to contributing records and upstream curation and transformation steps.
Which process led to a given model (e.g., the best-performing model)?	Final model	Traverses the full lineage from raw data through curation, preparation, and training to the final model.
Item history	Single data item $d_{i,a}$	Identifies downstream artifacts derived from this item, including transformations and dataset inclusion.
Impact on feature spread	Dataset D	Compares feature statistics before and after operations to detect shifts and potential bias.
Tracing records to metrics	Model M and dataset D	Traces evaluation metrics back to the specific training records and their preprocessing history.

segments, label inconsistencies in the annotation step, or an imbalanced sampling of non-challenging versus challenging intervals.

5.3 Queries Focused on Learning and Configurations

These queries favor both the learning data preparation and learning stages. Like so, they primarily address hyperparameter configurations, metrics, code, and environment settings captured during training and evaluation, or even during data learning preparation (e.g, the splitting ratio). Concretely, this information is typically recorded at run-level granularity. Returning to the ASD use case, the study evaluates multiple machine-learning techniques (SVM, DT, MLP, XGBoost) and different groups of feature sets to optimize classifier performance. From Table 3, the second query asks: "Across all experiment runs for the challenging behaviour prediction, which set of hyperparameters (e.g., XGBoost learning rate and tree depth) led to the highest average training accuracy?" Within our metamodel, this query is answered by traversing the retrospective execution history. First, the query targets all EvaluationMetric entities associated with training runs and selects those with the maximal (or otherwise optimal) metric value. Subsequently, once the best EvaluationMetric is identified, the traversal follows the wasGeneratedBy relationship to the corresponding StepRun instance (i.e., the training execution). In our design, the StepRun object carries the relevant configuration details and attributes (e.g., the specific optimization algorithm and number of epochs), so the query can simply inspect these attributes to recover the full hyperparameter setting used in that run. In this way, the provenance graph provides a principled, reproducible explanation of which training configuration achieved the best performance and supports systematic tuning and model selection for subsequent deployment in the ASD pipeline.

Table 3: Provenance queries focusing on learning and configuration stages during model training.

Provenance Query	Input / Focus	Alignment with End-to-End Pipeline
Which hyperparameters were used in this model?	Trained model	Retrieves the hyperparameters applied during the training StepRun that produced the final model.
Which hyperparameter configuration produced the highest training accuracy?	Metric (<i>accuracy</i>)	Identifies the run with the highest accuracy and traces back to its associated hyperparameter configuration.
Which change set resulted in model degradation?	Metric (<i>e.g., accuracy</i>)	Traces degraded performance to the corresponding training StepRun and its recorded file_revision.
What was the computational environment used to train a given model?	Trained model	Retrieves environment details (e.g., library versions, system metrics, hardware) linked to the training execution.

6 Implementation

In this section, we detail how the meta-model presented in Section 3 is implemented within a Python-based ecosystem. This is achieved through the construction of a unifying ETL layer that integrates DPDS for fine-grained provenance and MLflow/MLflow2prov for experimental tracking. As a result, the integrated provenance is materialized as a single graph in Neo4j and queried using Cypher, thereby enabling the query patterns presented in Tables 1, 2, and 3. From an operational standpoint, upon executing the preprocessing pipelines, DPDS observes state changes in panda DataFrames, where preprocessing scripts operate on tabular representations derived from raw sensor streams after resampling heterogeneous sampling rates and generating fixed-length windows.

The output of the DPDS layer consists of provenance records composing : (i) activities corresponding to concrete preprocessing executions (represented by StepRuns in our meta model), (ii) entities representing dataset elements at multiple levels of granularity, and (iii) relationships (usage, generations, and derivations) linking outputs to their contributing sources. In line with this design, the resulting provenance outputs reflect directly the meta-model’s fine-grained artifact hierarchy, namely DatasetArtifact, DataRecordArtifact, and DataCellArtifact, thus enabling cell-level and record-level lineage queries (Table 1). Moreover, our implementation materializes only changed/derived elements and their dependencies, avoiding the creation of redundant nodes for unmodified values. Similarly, an equivalent layer for coarse-grained capture of learning and evaluation experiments is also catered for. This particularly integrates MLflow for experiment tracking and MLflow2PROV.

By abstracting to this level, merging the disparate outputs from the above two layers into a unified graph would ultimately require a third integration layer. This integration process comprises two key steps. First, normalization is applied:

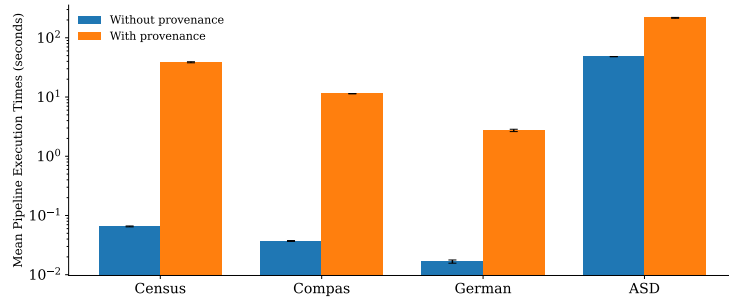


Fig. 3: Preprocessing pipeline execution time with and without provenance capture for Census, COMPAS, German, and ASD datasets. Bars report mean execution time over three runs ($\pm$ standard deviation). A logarithmic y-axis is used to accommodate the wide range of runtimes.

for the sake of consistency, both source graphs are converted into a common uniform node-edge representation that adheres to W3C PROV semantics and conforms to our meta-model classes. Second, cross-stage linking is performed. As explained in Section 4.2, the integration layer relies on a stable `record_id` attribute propagated from the Data Curation stage, where it serves as the primary identifier for record-level provenance (and, by composition, for the cells belonging to each record), through the Learning Data Preparation stage. The provenance graph obtained as a result is stored in a Neo4j graph database.

7 Experimental Evaluation

To assess the feasibility and efficiency of the proposed unified provenance meta-model, we conducted a set of experiments using four ML pipelines. The evaluation concentrates on three core performance dimensions: (i) the computational overhead produced by the capture mechanism, (ii) the storage requirements for the generated provenance graphs, and (iii) the query performance of the unified Neo4j data store. All experiments were performed on a standard laptop (Dell. Precision 3581, Intel Core i7, 32 GB RAM) using four benchmark datasets: Census, COMPAS, German Credit, and the ASD behavioural detection dataset.

7.1 Provenance capture overhead

Fig. 3 shows the overhead of provenance capture using four pipelines. Two observations follow. First, provenance capture introduces a substantial relative overhead for the smaller benchmark pipelines, primarily because the baseline preprocessing time is very small, whereas provenance capture entails materialising fine-grained entities and relations and persisting them to the provenance store. Second, for the ASD pipeline the baseline execution time is already non-trivial due to sensor processing and dataset construction; consequently, the relative overhead is lower,

but the absolute overhead is higher. This is expected in fine-grained capture settings where provenance volume scales with the number of produced/derived data elements and the number of transformation steps that modify them.

7.2 Storage footprint of provenance graphs

Table 4 shows the characteristics of the input datasets of the pipelines, and the size of the provenance graph captured for the pipelines. It indicates that provenance size is not solely a function of the number of input records, but also reflects (i) the number and nature of preprocessing operations, (ii) the extent of feature expansion or record expansion introduced by transformations (e.g., encoding, windowing), and (iii) the degree of value-level change that triggers materialisation of new provenance entities. In particular, the ASD pipeline produces the largest provenance graph (761.7 MB), consistent with its higher record count (41,148) and multi-step transformation workflow over physiological time series. By contrast, the COMPAS provenance footprint is comparatively small (2.9 MB), despite having more features than some other datasets, suggesting that the executed transformations in that pipeline generate fewer fine-grained derivations and/or fewer materialised changes at the value level.

7.3 Query performance on the ASD unified provenance graph

Regarding provenance querying, we focused on the last pipeline (ASD) since it has the largest provenance graph. The queries used for evaluation were formulated as Cypher queries, and are aligned with the representative query patterns in Tables 1, 2, and 3.

Each query was executed three times and we report the mean runtime in milliseconds. Because the ASD provenance graph is large, and because some queries may return very large result sets,

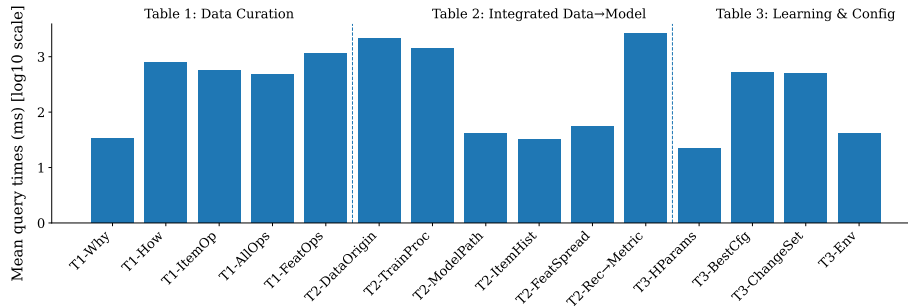


Fig. 4: Mean execution time of 15 Cypher provenance queries over the ASD unified provenance graph. Queries correspond to Tables 1, 2, and 3

Table 4: Input dataset characteristics and resulting provenance graph size.

Input Dataset	Records	Features	Provenance Graph Size (MB)
Census	32562	14	247.2
Compas	7215	52	2.9
German	1001	20	15
ASD	41148	22	761.7

we report query runtimes primarily for count-based or bounded outputs (e.g., returning counts of matching entities or short paths), which reflects a realistic interactive analysis pattern where users first locate relevant subgraphs before retrieving detailed sets of entities.

Fig. 4 demonstrates that query runtimes range from 21.66 ms to 2,63 ms, with a median of approximately 503 ms. The fastest queries are those that remain within run-level learning metadata (e.g., retrieving hyperparameters for a trained model), while the slowest are integrated traversals that link evaluation outcomes back to record-level derivations, as they require navigating larger regions of the graph (and, in some cases, aggregating over many records).

Overall, the empirical results reported above demonstrate that the proposed meta-model effectively supports provenance requirements across heterogeneous machine learning pipelines. Three main findings can be concluded from the evaluation. First, the execution-time overhead introduced by provenance capture depends on the nature of pipeline and is strongly influenced by the granularity of the provenance collected. In particular, fine-grained provenance capture incurs a measurable runtime overhead, which is most evident when applied to lightweight preprocessing pipelines such as those used for the Census, COMPAS, and German datasets. In these cases, baseline execution times are on the order of milliseconds; consequently, even modest absolute increases in execution time translate into substantial relative overheads. This behavior can be explained by the fact that baseline pipelines perform minimal data manipulation, whereas fine-grained provenance capture requires additional instrumentation, change detection, provenance object instantiation, and the materialization of provenance entities and relationships.

Second, the storage footprint of the provenance graph is affected by the characteristics of the dataset and, more importantly, by transformation patterns that significantly increase the number of derived elements, as observed in the ASD pipeline. Third, the unified provenance graph supports comprehensive and representative query classes across different stages of the pipeline. Query execution times range from milliseconds to a few seconds on the ASD workload, including queries that traverse links between fine-grained preprocessing provenance and model-level outcomes, thereby confirming the practical feasibility of cross-stage provenance analysis.

8 Conclusions

In this paper, we presented a model and an associated solution for capturing the provenance of ML pipelines, supporting both data preprocessing stages and model training and evaluation. The solution has been implemented and empirically evaluated, demonstrating its effectiveness in capturing and exploring both prospective and retrospective provenance. Ongoing work focuses on applying and assessing the approach on a broader range of pipelines, as well as illustrating how the captured provenance can be exploited for ML applications such as fairness analysis and debugging.

References

1. Alban, A.Q., Ayesh, M., Alhaddad, A.Y., Khalid Al-Ali, A., So, W.C., Connor, O., Cabibihan, J.J.: Detection of challenging behaviours of children with autism using wearable sensors during interactions with social robots
2. Arab, B.S., Feng, S., Glavic, B., Lee, S., Niu, X., Zeng, Q.: Gprom - A swiss army knife for your provenance needs. *IEEE Data Eng. Bull.* **41**(1), 51–62 (2018), <http://sites.computer.org/debull/A18mar/p51.pdf>
3. Belhajjame, K., B'Far, R., Cheney, J., Coppens, S., Cresswell, S., Gil, Y., Groth, P., Klyne, G., Lebo, T., McCusker, J., et al.: Prov-dm: The prov data model. *W3C Recommendation* **14**, 15–16 (2013)
4. Chapman, A., Lauro, L., Missier, P., Torlone, R.: Supporting better insights of data science pipelines with fine-grained provenance. *ACM Trans. Database Syst.* **49**(2), 6:1–6:42 (2024). <https://doi.org/10.1145/3644385>, <https://doi.org/10.1145/3644385>
5. Chen, A., Chow, A., Davidson, A., DCunha, A., Ghodsi, A., Hong, S.A., Konwinski, A., Mewald, C.: Developments in mlflow: A system to accelerate the machine learning lifecycle
6. Gharibi, G., Walunj, V., Nekadi, R., Marri, R., Lee, Y.: Automated end-to-end management of the modeling lifecycle in deep learning. *Empir. Softw. Eng.* **26**(2), 17 (2021)
7. Glavic, B., Alonso, G.: The perm provenance management system in action (2009)
8. Melgar, L.A., Dao, D., Gan, S., Gürel, N.M., Hollenstein, N., Jiang, J., Karlas, B., Lemmin, T.: Ease.ml: A lifecycle management system for machine learning
9. Pimentel, J.F., Freire, J., Murta, L., Braganholo, V.: A survey on collecting, managing, and analyzing provenance from scripts. *ACM Comput. Surv.* **52**(3), 47:1–47:38 (2019). <https://doi.org/10.1145/3311955>, <https://doi.org/10.1145/3311955>
10. Psallidas, F., Wu, E.: Smoke: Fine-grained lineage at interactive speed. *Proc. VLDB Endow.* **11**(6), 719–732 (2018). <https://doi.org/10.14778/3184470.3184475>, <http://www.vldb.org/pvldb/vol11/p719-psallidas.pdf>
11. Schlegel, M., Sattler, K.: Capturing end-to-end provenance for machine learning pipelines. *Inf. Syst.* **132**, 102495 (2025). <https://doi.org/10.1016/J.IS.2024.102495>, <https://doi.org/10.1016/j.is.2024.102495>
12. Souza, R., Azevedo, L.G., Lourenço, V., Soares, E.F.S., Thiago, R., Cerqueira, R., Netto, M.A.S.: Workflow provenance in the lifecycle of scientific machine learning
13. Vartak, M.: MODELDB: A system for machine learning model management. In: 8th Biennial Conference on Innovative Data Systems Research, CIDR 2017, Chaminade, CA, USA, January 8–11, 2017, Online Proceedings. www.cidrdb.org (2017)